Práctica 2:

Mi Agente en la Nube

Despliegue del Agente de Soporte con GitHub Actions

Objetivo

En la Práctica 1 el agente de soporte funcionaba solo mientras la computadora estaba encendida y el script corriendo en la terminal. En esta práctica el alumno llevará el agente a la nube usando **GitHub Actions**, un servicio gratuito de GitHub que ejecuta programas de forma automática en servidores de GitHub. Al terminar, el agente revisará el Google Sheet cada 30 minutos aunque la computadora del alumno esté apagada. Finalmente, se publicará la app de Google en modo de producción para que el agente funcione de forma permanente, sin que su acceso a Google caduque.

En el camino se aprenderán tres prácticas profesionales: separar los secretos (contraseñas y llaves) del código, versionar un proyecto con Git y automatizar su ejecución con un flujo de trabajo (*workflow*).

Funcionamiento del Agente en la Nube

La diferencia clave con la Práctica 1: el script ya no se queda en un bucle infinito esperando. GitHub Actions lo "despierta" cada 30 minutos, el script procesa las quejas pendientes y termina. El reloj ahora lo pone GitHub, no la librería *schedule*.

Requisitos Previos

- Haber terminado la **Práctica 1**: el agente debe funcionar en la computadora del alumno con *python agente_soporte.py*.
- Haber iniciado sesión con Google al menos una vez al ejecutar el agente (eso genera el archivo de sesión que se usará en la Fase 4).

Herramienta	Para qué sirve	Costo
Cuenta de GitHub	Guardar el código en un repositorio y ejecutar GitHub Actions	Gratuito
Git	Subir el código desde la computadora a GitHub (git-scm.com)	Gratuito
Visual Studio Code	Editar el script, el workflow y los archivos de configuración	Gratuito
Proyecto de la Práctica 1	Script, client_secrets.json, Google Form y Google Sheet	—

Conceptos Clave

Concepto	Qué es
Repositorio	Carpeta del proyecto guardada en GitHub, con el historial de todos sus cambios.
Commit / Push	Un *commit* guarda una versión del proyecto en la computadora; *push* la sube a GitHub.
Workflow	Archivo de instrucciones (formato YAML) que le dice a GitHub qué ejecutar y cuándo.
Runner	Computadora temporal de GitHub (Linux) donde se ejecuta el workflow. Se crea vacía en cada ejecución y se destruye al terminar.
Cron	Formato estándar para expresar horarios repetitivos, por ejemplo "cada 30 minutos".
Secret	Valor confidencial (API Key, token) que GitHub guarda cifrado y entrega al workflow como variable de entorno, sin que aparezca en el código.
Variable de entorno	Valor que el sistema operativo le pasa a un programa al ejecutarlo. En Python se lee con *os.getenv("NOMBRE")*.

FASE 1: Preparar el Script para la Nube

El script de la Práctica 1 tiene tres detalles que impiden subirlo a la nube: la API Key está escrita dentro del código, el inicio de sesión con Google necesita abrir un navegador (y el runner de GitHub no tiene pantalla), y el programa nunca termina por el bucle infinito. En esta fase se resuelven los tres.

Paso 1.1 — Sacar la API Key del Código

Cualquier valor secreto escrito en el código queda expuesto en cuanto el código se comparte. Por eso la API Key se moverá a un archivo aparte llamado **.env**, que nunca se sube a GitHub.

- Instalar la librería **python-dotenv**: *pip install python-dotenv*.
- En la carpeta del proyecto, crear un archivo llamado exactamente **.env** (con punto al inicio y sin extensión) con una sola línea:

```
GEMINI_API_KEY=AIzaSyAbc123xyzEjemploDeClaveReal456
```

- En el script, importar *load_dotenv* desde la librería *dotenv* y llamarla al inicio de la sección de configuración, para que cargue el contenido del **.env**.
- Cambiar la constante **GEMINI_API_KEY** para que, en lugar de tener la clave escrita, la lea con *os.getenv("GEMINI_API_KEY")*. Conviene darle como valor por defecto el texto de marcador de la Práctica 1, así la validación del punto de entrada sigue funcionando si la variable no existe.
- Crear también un archivo **.env.example** con la misma línea, pero con el texto de marcador en lugar de la clave real. Este sí se sube a GitHub, como plantilla para quien descargue el proyecto.

Concepto: En la computadora, *load_dotenv()* toma la clave del archivo .env. En GitHub Actions no existe ese archivo: la clave llegará como variable de entorno desde un Secret. Como en ambos casos se lee con *os.getenv()*, el mismo código funciona en los dos lugares.

Paso 1.2 — Iniciar Sesión con Google sin Navegador

Cuando el alumno inició sesión en la Práctica 1, Google entregó un **token** que se guardó en el archivo *authorized_user.json*. Ese token incluye un *refresh token* que permite obtener acceso nuevo sin volver a abrir el navegador. En GitHub Actions el contenido de ese archivo llegará en la variable de entorno **GOOGLE_TOKEN_JSON**.

- Agregar las importaciones **Credentials** (de *google.oauth2.credentials*) y **Request** (de *google.auth.transport.requests*).
- Al inicio de la función **conectar_sheets()**, antes de cualquier otra cosa, leer la variable de entorno *GOOGLE_TOKEN_JSON*.
- Si la variable existe: convertir su texto a diccionario con *json.loads()*, crear las credenciales con *Credentials.from_authorized_user_info()* (pasándole el diccionario y los mismos scopes de la Práctica 1), renovarlas llamando a su método *refresh()* con un objeto *Request()*, y finalmente autorizar gspread con *gspread.authorize(creds)*, abrir la hoja por su nombre y retornar *sheet1*.
- Si la variable no existe: continuar exactamente como en la Práctica 1 (inicio de sesión con el navegador). Así el script sigue funcionando igual en la computadora.

Paso 1.3 — Agregar el Modo "Una Vez"

GitHub Actions no puede tener un programa corriendo para siempre: cada ejecución tiene tiempo límite y consume minutos de la cuota gratuita. El script debe poder ejecutarse una sola vez y terminar.

- Importar el módulo **sys**, que permite leer los argumentos con los que se ejecutó el script (la lista *sys.argv*).
- Modificar **ejecutar_agente()** para que **retorne False** si no se pudo conectar a Google Sheets, y **True** al terminar el ciclo normalmente.
- En el punto de entrada, después de inicializar el cliente de Gemini y antes de configurar *schedule*: si el texto **--una-vez** está en *sys.argv*, llamar a *ejecutar_agente()* una sola vez y terminar el programa con *exit(0)* si tuvo éxito o con *exit(1)* si falló.
- Si no se pasa *--una-vez*, el script se comporta como en la Práctica 1 (bucle infinito).

Concepto: El número que se pasa a *exit()* es el **código de salida**. Por convención, 0 significa "todo bien" y cualquier otro número significa error. GitHub lo usa para marcar la ejecución en verde o en rojo, y envía un correo cuando una ejecución falla.

Paso 1.4 — Ajustar la Validación de Credenciales

En el punto de entrada, el script verifica que exista *client_secrets.json*. Ese archivo no se subirá a GitHub, así que la validación debe aplicarse **solo cuando no exista** la variable de entorno *GOOGLE_TOKEN_JSON*.

Paso 1.5 — Probar en la Computadora

Ejecutar el script en modo "una vez" desde la terminal:

```
python agente_soporte.py --una-vez
```

El agente debe procesar las filas pendientes (si hay), mostrar el resumen de filas procesadas y **regresar solo a la terminal**, sin quedarse esperando. Probar también *python agente_soporte.py* sin el argumento para confirmar que el modo normal sigue funcionando (detenerlo con Ctrl+C).

FASE 2: Preparar los Archivos del Repositorio

Paso 2.1 — Crear el Archivo requirements.txt

El runner de GitHub se crea vacío en cada ejecución, así que hay que decirle qué librerías instalar. Crear un archivo **requirements.txt** con el nombre de cada librería en su propia línea: *gspread*, *google-auth*, *google-auth-oauthlib*, *google-genai*, *schedule* y *python-dotenv*.

Paso 2.2 — Crear el Archivo .gitignore

El archivo **.gitignore** le indica a Git qué archivos **nunca** debe subir. Crearlo en la carpeta del proyecto con una línea por cada archivo a excluir:

- **.env** — contiene la API Key de Gemini.
- **client_secrets.json** — contiene las credenciales OAuth de la aplicación.
- **__pycache__/** — carpeta de archivos temporales que genera Python.

Importante: Si por error se sube la API Key a GitHub, borrarla del repositorio **no es suficiente**: queda guardada en el historial. Hay que entrar a Google AI Studio, eliminar esa clave y generar una nueva. Existen programas que buscan claves expuestas en GitHub en minutos.

Paso 2.3 — Verificar la Organización de Archivos

Al terminar la Fase 2, la carpeta del proyecto debe verse así:

```
mi_agente_soporte/
  agente_soporte.py      <-- modificado en la Fase 1
  requirements.txt       <-- se sube a GitHub
  .gitignore             <-- se sube a GitHub
  .env.example           <-- se sube a GitHub (plantilla sin la clave real)
  .env                   <-- NO se sube (lo excluye .gitignore)
  client_secrets.json    <-- NO se sube (lo excluye .gitignore)
```

FASE 3: Subir el Proyecto a GitHub

Paso 3.1 — Crear la Cuenta e Instalar Git

- Crear una cuenta en **github.com** (si aún no se tiene). Con el correo institucional se puede solicitar el **GitHub Student Developer Pack** en *education.github.com*, que da beneficios adicionales, pero no es obligatorio para esta práctica.
- Descargar e instalar Git desde **git-scm.com** con las opciones predeterminadas.

- Abrir una terminal nueva y configurar el nombre y correo que aparecerán en los commits:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_correo@ejemplo.com"
```

Paso 3.2 — Crear el Repositorio en GitHub

- En GitHub, hacer clic en el botón **"+"** (arriba a la derecha) → **"New repository"**.
- Nombre del repositorio: **agente-soporte**.
- Seleccionar **Private** (privado).
- **No** marcar las opciones de agregar README, .gitignore ni licencia: el repositorio debe crearse vacío. Hacer clic en **"Create repository"**.
- Copiar la URL que muestra GitHub, con la forma *https://github.com/usuario/agente-soporte.git*.

Nota: El repositorio debe ser **privado** porque los registros de cada ejecución muestran los nombres y las quejas que llegan al formulario. En un repositorio público cualquier persona podría leerlos.

Paso 3.3 — Subir el Código

En la terminal, dentro de la carpeta del proyecto, ejecutar los siguientes comandos uno por uno:

```
git init
git add .
git status
```

El comando *git status* muestra la lista de archivos que se van a subir. **Revisarla con cuidado:** no deben aparecer *.env* ni *client_secrets.json*. Si aparecen, revisar el .gitignore antes de continuar. Si todo está bien, crear el commit y subirlo:

```
git commit -m "Agente de soporte listo para GitHub Actions"
git branch -M main
git remote add origin https://github.com/usuario/agente-soporte.git
git push -u origin main
```

- La primera vez que se haga *push*, se abrirá una ventana para iniciar sesión en GitHub.
- Al terminar, recargar la página del repositorio en GitHub y confirmar que aparecen los archivos, y que **.env** y **client_secrets.json** no están.

FASE 4: Guardar los Secretos en GitHub

El agente necesita dos valores confidenciales que no están en el repositorio: la API Key de Gemini y el token de Google. Se guardarán como **Secrets** del repositorio. Una vez guardado, un secret no se puede volver a ver, ni siquiera por el dueño; solo se puede reemplazar. GitHub además oculta su valor con asteriscos si llegara a imprimirse en los registros.

Paso 4.1 — Abrir la Sección de Secrets

- En el repositorio, ir a **Settings** → **Secrets and variables** → **Actions**.
- Para cada secret, hacer clic en **"New repository secret"**, escribir el nombre **exactamente** como se indica (mayúsculas incluidas), pegar el valor y hacer clic en **"Add secret"**.

Paso 4.2 — Secret GEMINI_API_KEY

- Nombre: **GEMINI_API_KEY**.
- Valor: la API Key de Gemini, tal como está en el archivo .env (solo la clave, sin *GEMINI_API_KEY=* ni comillas).

Paso 4.3 — Secret GOOGLE_TOKEN_JSON

- Abrir el Explorador de archivos y escribir en la barra de dirección: **%APPDATA%\gspread**
- Abrir el archivo **authorized_user.json** con el Bloc de notas, seleccionar **todo** el contenido (Ctrl+A) y copiarlo.
- Nombre del secret: **GOOGLE_TOKEN_JSON**. Valor: el contenido completo copiado, incluyendo las llaves *{ }* del inicio y del final.

Importante: Mientras la app de Google Cloud esté en modo **Testing** (como se configuró en la Práctica 1), el token deja de funcionar a los **7 días** de haber iniciado sesión. Si el archivo authorized_user.json tiene más de unos días, conviene renovarlo antes de copiarlo: borrar el archivo, ejecutar el agente en la computadora para iniciar sesión de nuevo, y copiar el archivo recién creado. En la Fase 7 se eliminará este límite.

FASE 5: Escribir el Workflow de GitHub Actions

El workflow es un archivo de texto en formato **YAML** que describe cuándo y cómo ejecutar el agente. GitHub lo detecta automáticamente si está en la ruta correcta.

Paso 5.1 — Crear el Archivo

- Dentro de la carpeta del proyecto, crear la carpeta **.github** (con punto al inicio) y dentro de ella otra carpeta llamada **workflows**.
- Dentro de *workflows*, crear el archivo **agente.yml**. La ruta completa debe ser **.github/workflows/agente.yml**; si el nombre de alguna carpeta está mal escrito, GitHub ignorará el archivo.

Nota: En YAML la **sangría** (los espacios al inicio de cada línea) define la estructura, igual que en Python. Se deben usar espacios, nunca tabuladores, y ser consistentes (se recomiendan 2 espacios por nivel). La documentación oficial está en *docs.github.com/actions*.

Paso 5.2 — Contenido del Workflow

El archivo debe contener las siguientes secciones:

- **name:** el nombre con el que aparecerá en GitHub, por ejemplo *Agente de Soporte*.
- **on:** los eventos que disparan el workflow. Se necesitan dos: *schedule*, con una entrada *cron* que lo ejecute cada 30 minutos, y *workflow_dispatch*, que agrega un botón para ejecutarlo manualmente desde GitHub (muy útil para probar).
- **jobs:** un único trabajo (por ejemplo *ejecutar-agente*) que corra en *ubuntu-latest* y tenga un límite de tiempo (*timeout-minutes*) de 10 minutos, para que un error no consuma la cuota.
- **steps:** los pasos del trabajo, en este orden:

#	Paso	Qué debe hacer
1	Descargar el código	Usar la acción *actions/checkout@v4*, que copia el repositorio al runner.

#	Paso	Qué debe hacer
2	Instalar Python	Usar la acción *actions/setup-python@v5* indicando la versión *"3.12"*.
3	Instalar dependencias	Ejecutar (*run*) el comando *pip install -r requirements.txt*.
4	Ejecutar el agente	Ejecutar *python agente_soporte.py --una-vez*, definiendo en su sección *env* las variables GEMINI_API_KEY y GOOGLE_TOKEN_JSON a partir de los secrets.

Para leer un secret dentro del workflow se usa la sintaxis **${{ secrets.NOMBRE_DEL_SECRET }}**. Por ejemplo, la variable GEMINI_API_KEY del paso 4 debe tomar el valor *${{ secrets.GEMINI_API_KEY }}*.

Paso 5.3 — Entender la Expresión Cron

Una expresión cron tiene 5 campos separados por espacios. El asterisco (*) significa "cualquier valor" y **/N* significa "cada N". Para ejecutar cada 30 minutos se usa **"*/30 * * * *"** (entre comillas en el archivo YAML).

Campo	Significado	Valor en "*/30 * * * *"
1	Minuto (0-59)	*/30 → en el minuto 0 y en el 30
2	Hora (0-23, en hora UTC)	* → todas las horas
3	Día del mes (1-31)	* → todos los días
4	Mes (1-12)	* → todos los meses
5	Día de la semana (0-6, 0 = domingo)	* → todos los días de la semana

Nota: ¿Por qué cada 30 minutos y no cada 5? En repositorios privados, el plan gratuito de GitHub da **2,000 minutos de ejecución al mes**, y cada ejecución cuenta como mínimo 1 minuto. Cada 30 minutos son unas 1,440 ejecuciones al mes y caben en la cuota; cada 15 minutos serían unas 2,880 y se agotaría antes de fin de mes.

Paso 5.4 — Subir el Workflow

Guardar el archivo y subirlo a GitHub con los comandos:

```
git add .
git commit -m "Agregar workflow de GitHub Actions"
git push
```

FASE 6: Ejecutar y Verificar en la Nube

Paso 6.1 — Primera Ejecución Manual

- En el repositorio, ir a la pestaña **Actions**. En la lista de la izquierda debe aparecer el workflow **Agente de Soporte**. Si no aparece, revisar la ruta del archivo (Paso 5.1).
- Seleccionarlo, hacer clic en **"Run workflow"** → **"Run workflow"**.
- Después de unos segundos aparecerá una ejecución nueva. Hacer clic en ella y luego en el trabajo *ejecutar-agente* para ver los registros de cada paso.

Si todo está correcto, la ejecución terminará con una **palomita verde** en menos de un minuto. En los registros del paso "Ejecutar el agente" debe verse la misma salida que en la terminal de la computadora: el encabezado de ejecución, el análisis de cada fila nueva (si había alguna) y el resumen con el número de filas ya procesadas y nuevas. Si termina con una **X roja**, abrir el paso que falló y consultar la tabla de errores al final de este documento.

Paso 6.2 — Prueba con la Computadora Apagada

Esta es la prueba definitiva de que el agente vive en la nube:

- Enviar al menos dos quejas nuevas desde el Google Form (pueden reutilizarse los ejemplos de la Práctica 1).
- **Cerrar la terminal y apagar o suspender la computadora.**
- Esperar entre 30 y 45 minutos y revisar el Google Sheet desde el celular: las columnas D, E y F de las quejas nuevas deben estar llenas.
- En la pestaña Actions debe aparecer una ejecución con el evento **schedule** (programada), además de la manual.

Nota: Las ejecuciones programadas de GitHub no son exactas: dependiendo de la carga de los servidores pueden retrasarse varios minutos. La primera ejecución programada puede tardar en aparecer después de subir el workflow. Para probar al momento, siempre se puede usar "Run workflow".

FASE 7: Publicar la App de Google en Producción

El agente ya funciona en la nube, pero tiene fecha de caducidad: mientras la app de Google Cloud esté en modo **Testing**, el token deja de funcionar a los 7 días y el workflow empezará a fallar con el error *invalid_grant*. Para que el agente funcione de forma permanente, la app debe cambiar al modo **En producción**.

Para permitirlo, Google exige que la app tenga una **página principal** y una **política de privacidad** publicadas en internet. En esta fase se crearán ambas páginas con **GitHub Pages**, un servicio gratuito de GitHub para publicar sitios web estáticos.

Paso 7.1 — Crear el Repositorio de las Páginas

Las páginas deben ser públicas, pero el repositorio del agente es privado (y GitHub Pages no está disponible para repositorios privados en el plan gratuito). Por eso se usará un segundo repositorio, solo para las páginas:

- En GitHub, crear un repositorio nuevo llamado **agente-soporte-info**, esta vez **Public** (público), y marcar la opción **"Add a README file"** para que no se cree vacío.
- Dentro del repositorio, usar **Add file** → **Create new file** para crear los dos archivos HTML que se describen en los pasos 7.2 y 7.3. También pueden crearse en la computadora y subirse con Git, como en la Fase 3.

Importante: Todo lo que se escriba en este repositorio será visible para cualquier persona. No incluir datos de los clientes, la API Key, ni el contenido de ningún archivo de credenciales.

Paso 7.2 — Página Principal (index.html)

Crear el archivo **index.html**: una página HTML sencilla que presente la app. Debe incluir:

- El **nombre de la app**, igual al que se usará en Google Cloud (por ejemplo, *Agente de Soporte IA*).
- **Qué hace:** revisa las respuestas de un formulario guardadas en Google Sheets, usa Gemini para clasificar cada queja (Ventas, Soporte Técnico o Logística) y detectar su sentimiento, y escribe el resultado en la misma hoja.
- **Para qué existe:** es un proyecto escolar del taller de automatización con IA.

- **Quién la usa:** únicamente el alumno, con su propia cuenta de Google; no es un servicio abierto al público.
- Un **enlace** a la página de política de privacidad (*privacidad.html*).

Paso 7.3 — Política de Privacidad (privacidad.html)

Crear el archivo **privacidad.html**, con la fecha de última actualización y las siguientes secciones:

- **Datos a los que accede:** solo la hoja de cálculo del proyecto; lee el nombre y el comentario de cada respuesta y escribe la clasificación, el sentimiento y la marca de procesado.
- **Cómo usa los datos:** el texto de cada comentario se envía a la API de Google Gemini únicamente para analizarlo, y los resultados se guardan solo en la hoja original.
- **Lo que no hace:** no guarda datos en otros servidores, no los vende ni los comparte con terceros (salvo el envío a Gemini), no los usa con fines publicitarios y no accede a otros archivos de Google Drive.
- **Cómo revocar el acceso:** desde *myaccount.google.com/permissions*.
- **Uso de datos de Google:** una declaración de que el uso de la información recibida de las APIs de Google se apega a la *Google API Services User Data Policy*, incluidos los requisitos de uso limitado (*Limited Use*).

Paso 7.4 — Activar GitHub Pages

- En el repositorio *agente-soporte-info*, ir a **Settings** → **Pages**.
- En **Source**, seleccionar **"Deploy from a branch"**; en **Branch**, elegir **main** y la carpeta **/ (root)**. Hacer clic en **Save**.
- Esperar uno o dos minutos y recargar la página. Arriba aparecerá la dirección del sitio, con la forma *https://usuario.github.io/agente-soporte-info/*.
- Abrir en el navegador las dos direcciones y confirmar que ambas cargan correctamente:

```
https://usuario.github.io/agente-soporte-info/
https://usuario.github.io/agente-soporte-info/privacidad.html
```

Paso 7.5 — Configurar la Marca de la App en Google Cloud

- En Google Cloud Console, con el proyecto *Agente de Soporte* seleccionado, ir a **APIs y servicios** → **Pantalla de consentimiento de OAuth** y abrir la sección **Desarrollo de la marca** (*Branding*).
- Llenar los campos como se indica en la tabla y hacer clic en **Guardar**:

Campo	Qué escribir
Nombre de la app	El mismo nombre usado en la página principal
Correo electrónico de asistencia	El correo del alumno
Logotipo de la app	**Dejarlo vacío** (ver nota)
Página principal de la aplicación	https://usuario.github.io/agente-soporte-info/
Vínculo a la Política de Privacidad	https://usuario.github.io/agente-soporte-info/privacidad.html
Vínculo a las Condiciones del Servicio	Dejarlo vacío (es opcional)
Dominios autorizados	usuario.github.io

Campo	Qué escribir
Información de contacto del desarrollador	El correo del alumno

Nota: No subir logotipo. Si la app tiene logotipo, Google obliga a enviarla a un proceso de verificación que puede tardar semanas. Sin logotipo, la app se publica como "no verificada", lo cual es suficiente para uso personal.

Paso 7.6 — Publicar la App

- En la misma pantalla de consentimiento de OAuth, ir a la sección **Público** (*Audience*).
- En **Estado de publicación**, hacer clic en **"Publicar app"** y confirmar.
- El estado debe cambiar a **En producción**.

Nota: Si aparece el mensaje "Se requieren un nombre de app, un correo electrónico de asistencia, una URL de página principal y una URL de política de privacidad válidos...", significa que falta algún dato del Paso 7.5. Regresar a Desarrollo de la marca, completarlo y volver a intentar.

Paso 7.7 — Generar un Token Permanente

El token actual se creó cuando la app estaba en Testing, así que seguirá caducando. Hay que generar uno nuevo:

- Borrar el archivo **authorized_user.json** de la carpeta *%APPDATA%\gspread*.
- Ejecutar el agente en la computadora con *python agente_soporte.py --una-vez*. Se abrirá el navegador para iniciar sesión.
- Google mostrará el aviso **"Google no verificó esta app"**. Es normal, porque la app es del propio alumno: hacer clic en **Configuración avanzada** → **Ir a (nombre de la app)** y luego en **Continuar** para autorizar el acceso.
- Copiar el contenido completo del nuevo **authorized_user.json**, como en el Paso 4.3.

Paso 7.8 — Actualizar el Secret y Verificar

- En el repositorio del agente, ir a **Settings** → **Secrets and variables** → **Actions**.
- Hacer clic en el ícono del lápiz junto a **GOOGLE_TOKEN_JSON**, pegar el contenido nuevo y guardar con **"Update secret"**.
- En la pestaña **Actions**, ejecutar el workflow con **"Run workflow"** y confirmar que termina en verde.

Concepto: Reemplazar una credencial por una nueva se llama **rotar** un secreto. Gracias a que el token vive en un Secret y no en el código, se pudo cambiar sin modificar ni una línea del script ni del workflow.

Solución de Errores Comunes

Error	Causa probable	Solución
"Debes reemplazar GEMINI_API_KEY" en los registros de GitHub	El secret no existe o su nombre está mal escrito	Revisar en Settings → Secrets que se llame exactamente GEMINI_API_KEY
invalid_grant: Token has been expired or revoked	El token de Google caducó (7 días en modo Testing)	Borrar authorized_user.json, ejecutar el agente en la computadora y actualizar el secret GOOGLE_TOKEN_JSON
JSONDecodeError al leer GOOGLE_TOKEN_JSON	No se copió el contenido completo del archivo	Volver a copiar todo, incluyendo las llaves { } del inicio y del final
El workflow no aparece en la pestaña Actions	El archivo no está en .github/workflows/	Revisar el nombre de las carpetas (con punto y en plural) y hacer push de nuevo
Invalid workflow file	Error de sangría o de sintaxis en el YAML	Revisar que se usen espacios (no tabuladores) y la sangría de cada sección
ModuleNotFoundError en GitHub	Falta una librería en requirements.txt	Agregar la librería que indica el error y hacer push
git push pide usuario y falla la contraseña	GitHub ya no acepta contraseñas en la terminal	Usar la ventana de inicio de sesión del navegador que abre Git para Windows
Aparece .env en git status	El .gitignore no existe, tiene otro nombre o se guardó como .gitignore.txt	Revisar el nombre exacto del archivo antes de hacer commit
Error 404 al abrir la página de GitHub Pages	Pages aún no termina de publicar, o el archivo no se llama exactamente index.html	Esperar unos minutos y revisar el nombre del archivo y la configuración del Paso 7.4
No aparece "Configuración avanzada" al iniciar sesión	La app sigue en Testing o el correo no corresponde	Revisar que el estado sea "En producción" e iniciar sesión con la misma cuenta del proyecto
invalid_grant después de la Fase 7	El secret aún tiene el token creado en modo Testing	Repetir los pasos 7.7 y 7.8

Comparación: Práctica 1 vs Práctica 2

Aspecto	Práctica 1 (computadora)	Práctica 2 (GitHub Actions)
¿Dónde corre?	En la computadora del alumno	En un runner de GitHub (Linux)
¿Quién lo repite?	La librería schedule + while True	El cron del workflow
¿Cuánto dura?	Corre sin parar hasta Ctrl+C	Procesa pendientes y termina (--una-vez)
API Key de Gemini	Archivo .env	Secret GEMINI_API_KEY
Sesión de Google	Navegador + authorized_user.json	Secret GOOGLE_TOKEN_JSON
Duración del token	7 días (app en Testing)	Permanente (app en producción)
¿Y si falla?	Se ve en la terminal	X roja en Actions y aviso por correo

Entregables

- **Acceso al repositorio:** agregar al profesor como colaborador en **Settings** → **Collaborators** → **Add people**, usando el usuario de GitHub que indique el profesor, y entregar la URL del repositorio.
- **Captura 1:** la pestaña Actions con al menos una ejecución manual y una programada (*schedule*) terminadas en verde.
- **Captura 2:** los registros del paso "Ejecutar el agente" de una ejecución que haya procesado al menos una queja nueva.
- **Captura 3:** el Google Sheet con las quejas de la prueba del Paso 6.2 ya clasificadas por el agente.
- **URLs de GitHub Pages:** la página principal y la política de privacidad del Paso 7.4.
- **Captura 4:** la sección Público (*Audience*) de Google Cloud con el estado **En producción**.
- **Captura 5:** una ejecución en verde posterior a la actualización del secret del Paso 7.8.
- **Respuestas** a las preguntas de reflexión.

Preguntas de Reflexión

- ¿Por qué el archivo .env no se sube a GitHub, pero el .env.example sí?
- ¿Por qué el agente necesita el modo *--una-vez* en GitHub Actions, si en la computadora funcionaba bien con el bucle infinito?
- Si la app se hubiera quedado en modo Testing, ¿qué le pasaría al agente dos semanas después? ¿Cómo se enteraría el alumno?
- ¿Por qué Google exige una política de privacidad para publicar la app? ¿Qué datos de los usuarios maneja este agente?
- Si el repositorio fuera público, ¿qué información de los clientes quedaría expuesta y dónde?
- Calcular cuántos minutos de la cuota gratuita consumiría el agente al mes si se ejecutara cada 10 minutos. ¿Cabría en los 2,000 minutos?